

ATENÇÃO: TODOS OS PROGRAMAS FORAM COMPILADOS NO UBUNTU 24.04 COM O G++ NA VERSÃO 13.3.0.

1 Programa A-exemploIPC0.cpp

1.1 Compile o código, execute o programa várias vezes e anote o seu resultado.

1.2 Altere o código para forçar uma condição de corrida. Compile-o, execute-o várias vezes e anote o seu resultado.

1.3 Explique o que você fez no código e explique porque está ocorrendo a condição de corrida.

2 Programa B-exemploIPC2.cpp

2.1 Qual é o método de IPC empregado para coordenar as atividades das duas threads? Embase sua resposta fazendo referência ao código.

2.2 Compile o código e mantenha o programa em execução por um tempo.

2.3 Analise o resultado da execução do código. As threads thread1() e thread2() estão entrando na região crítica ao mesmo tempo? Explique como chegou a essa conclusão.

2.4 Mantendo o programa ainda em execução, abra outra janela de terminal e entre no utilitário htop do Linux. Como saber, na prática, se o método de IPC empregado é um método de espera ocupada? Explique.

2.5 O código está fazendo uso de alguma API do sistema operacional para realizar IPC? Explique.

3 Programa C-exemploIPC6.cpp

3.1 Compile o código e mantenha o programa em execução por um tempo.

3.2 Usando o utilitário htop, o que se percebeu de diferença ao comparar a execução deste programa com o programa anterior? Explique.

3.3 Por que a linha 22 está comentada e não incorre em nenhum erro na execução do programa? Explique.

3.4 O programa faz uso de cinco threads produtoras e duas threads consumidoras. No decorrer da execução do programa, contudo, o buffer fica vazio. Por quê? Explique, fazendo referência ao código-fonte.

3.5 Que alteração no código poderia ser feita para que o buffer fique cheio no decorrer da execução do código?

4 Programas D-somavetor1.cpp e D-somavetor2.cpp

4.1 Compile os dois códigos-fonte e execute os dois programas, um por vez. Atenção: um dos programas é *monothread*. O outro, *multithread*. Assim, anote o tempo de execução do *monothread* e diversos tempos de execução do *multithread* com variadas quantidades de *threads*.

4.2 Por que há diferença no tempo de execução do programa *monothread* vs *multithread*?

4.3 Que alteração você faria para deixar o código *multithread* mais performático?

4.4 Depois que você alterou o código *multithread* para deixá-lo mais rápido, responda à seguinte pergunta: para o código *multithread* em questão e para o seu computador, qual é a quantidade ideal de threads para executar este programa?

4.5 Altere o prompt de compilação que você usou, tirando o parâmetro -O3, e anote os tempos de execução dos dois programas. Por que houve alteração nos tempos de execução dos dois programas?